

ЛАБОРАТОРНАЯ РАБОТА №10
РАЗРАБОТКА ИНТЕРФЕЙСА ПРИЛОЖЕНИЯ HANGMAN
Что делаем сегодня?

Сегодня мы сделаем **интерфейс** нашего приложения “Виселица”, а в ЛР11 напишем саму **логику** – правила, слово, проверку букв.

Интересно: сегодня мы не создадим в XML **ни одной кнопки** – все 32 клавиши появятся из кода циклом. К концу пары поймёте, почему это лучше, чем ~200 строк XML.

Основные правила

1. Лабораторная работа разделена на **ЧАСТИ** – выполняете каждую из частей и **показываете**.
2. Обычно в конце каждой части есть **ЗАДАНИЕ** – его надо сделать
3. Не бойтесь **задавать любые** вопросы
4. Если хотите сделать что-то **свое**, то **предлагайте** – будем думать
5. Советую читать текст, тут **написано** много полезного
- 6.



Часть 1. Разметка

Полный файл **activity_main.xml** скопируйте его отсюда либо - <https://github.com/aiinty/labs> (полностью набирать XML руками не будем):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:id="@+id/main"
    android:padding="8dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Виселица"
        android:textSize="22sp"
        android:textStyle="bold" />

    <!--
        ЗАМЕСТО ЭТОГО КОММЕНАТРИЯ БУДЕТ ВАШ HangmanView
    -->

    <!-- МАСКА слова -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="_ _ _ _ _"
        android:textSize="30sp"
        android:typeface="monospace" />

    <!-- СТАТУС игры -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Осталось попыток:"
        android:textSize="16sp" />

    <!-- 1 ряд кнопок клавиатуры -->
    <LinearLayout
        android:id="@+id/row1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:orientation="horizontal" />

    <!-- 2 ряд кнопок клавиатуры -->
    <LinearLayout
        android:id="@+id/row2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal" />

    <!-- 3 ряд кнопок клавиатуры -->
    <LinearLayout
        android:id="@+id/row3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal" />
```

```

<!-- 4 ряд кнопок клавиатуры -->
<LinearLayout
    android:id="@+id/row4"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Новая игра" />
</LinearLayout>

```

1. Обратите внимание, там есть:

```
android:layout_height="0dp"
```

и

```
android:layout_weight="1"
```

мы так используем вес внутри **LinearLayout**. Понять это можно так:

“высота **НОЛЬ** по умолчанию, но отдай **ВЫСОТЕ** всё свободное место по вертикали”.

2. Обратите внимание, мы используем:

```
android:typeface="monospace"
```

это делает **МОНОШИРИННЫЙ** шрифт: у всех символов одинаковая ширина, поэтому “_ _ _ _” выглядит ровно.

! ЗАДАНИЕ !

1. вынесите все строки из элементов в ресурсы **strings.xml**.
2. назначьте каждому элементу свой **id**, например:

```
android:id="@+id/btnOrder"
```

Часть 2. Оживляем кнопку: слушатели

Всё рисование вы уже умеете из ЛР4. Новое здесь только организация: рисунок состоит из этапов, и этапами можно управлять извне.

Создайте класс **HangmanView** (наследник **View**, выбираете **второй** конструктор – всё как в ЛР4):

```
public class HangmanView extends View {  
  
    public static final int MAX_STAGES = 6; // максимум ошибок 3 usages  
    private int stage = 0; // текущий этап: 0 - только виселица, 6 - человечек ГОТОВ no usages  
    private Paint paint; 6 usages  
  
    public HangmanView(Context context, AttributeSet attrs) {  
        super(context, attrs);  
  
        paint = new Paint();  
        paint.setColor(Color.DKGRAY);  
        paint.setStyle(Paint.Style.STROKE);  
        paint.setStrokeWidth(10f);  
        paint.setStrokeCap(Paint.Cap.ROUND);  
        paint.setAntiAlias(true);  
    }  
}
```

Далее. Сделаем чтоб рисовалась сама **стойка** –

```
@Override  
protected void onDraw(Canvas canvas) {  
    super.onDraw(canvas);  
    drawScaffold(canvas);  
}  
  
private void drawScaffold(Canvas c) { 1 usage  
    float w = getWidth();  
    float h = getHeight();  
  
    float groundY = h - 100;  
    float poleX = w * 0.25f;  
    float topY = 100;  
    float ropeX = w * 0.62f;  
  
    // земля  
    c.drawLine(startX: w * 0.15f, groundY, stopX: w * 0.65f, groundY, paint);  
  
    // стойка  
    c.drawLine(poleX, groundY, poleX, topY, paint);  
  
    // перекладина  
    c.drawLine(poleX, topY, ropeX, topY, paint);  
  
    // верёвка  
    c.drawLine(ropeX, topY, ropeX, stopY: topY + 80, paint);  
}
```

Если хотите, то можете попробовать запустить – у вас **нарисуется** пустая стойка.
(не забудьте добавить ваш **HangmanView** в **xml** вашего экрана)

Теперь. Сделаем чтоб **человечек** рисовался по этапам. **Каждая ошибка игрока добавляет одну деталь.** Логика простая: “если этап уже достиг N – часть N нарисована”:

```
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);
    drawScaffold(canvas);
    drawStickman(canvas);
}

private void drawScaffold(Canvas c) {...}

private void drawStickman(Canvas c) {
    float x = getWidth() * 0.62f;
    float y = getHeight() * 0.21f;

    if (stage >= 1)
        c.drawCircle(x, y, radius: 40, paint);

    if (stage >= 2)
        c.drawLine(x, startY: y + 40, x, stopY: y + 250, paint);

    if (stage >= 3)
        c.drawLine(x, startY: y + 80, stopX: x - 70, stopY: y + 150, paint);

    if (stage >= 4)
        c.drawLine(x, startY: y + 80, stopX: x + 70, stopY: y + 150, paint);

    if (stage >= 5)
        c.drawLine(x, startY: y + 250, stopX: x - 60, stopY: y + 400, paint);

    if (stage >= 6)
        c.drawLine(x, startY: y + 250, stopX: x + 60, stopY: y + 400, paint);
}
```

(метод drawScaffold я свернул – но он все еще есть)

Что делаем теперь? – сделаем так чтоб можно было изменять наш HangmanView с экрана (то есть снаружи – не внутри HangmanView) и заставим человечка нарисоваться.

Сейчас этапом (у нас в классе есть **stage**) никто не управляет – добавим временный способ проверки: тап по холсту переключает этап. Заодно добавим метод, через который в ЛРП будем человечком – допишем HangmanView:

```
// МЕТОД который будем вызывать из MainActivity
// чтобы менять этап в ОБЪЕКТЕ HangmanView
public void setStage(int stage) { 2 usages
    if (stage < 0) stage = 0; // проверка на отрицательное число
    if (stage > MAX_STAGES) stage = MAX_STAGES; // если превышаем максимум, то больше не делаем
    this.stage = stage; // заносим изменения в ОБЪЕКТ
    postInvalidate();
}

// ВРЕМЕННО: нажатие переключает этап
@Override
public boolean onTouchEvent(MotionEvent event) {
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
        stage++; // меняем этап на 1 больше
        postInvalidate();

        Log.d(tag: "Hangman", msg: "Этап: " + stage);
        return true;
    }
    return super.onTouchEvent(event);
}
```

Запустите и понажимайте: каждый тап добавляет деталь, после шестого человечек готов. В **Logcat** (фильтр tag:Hangman) видны номера этапов.

Часть 3. Создаем в коде клавиатуру

Для начала –

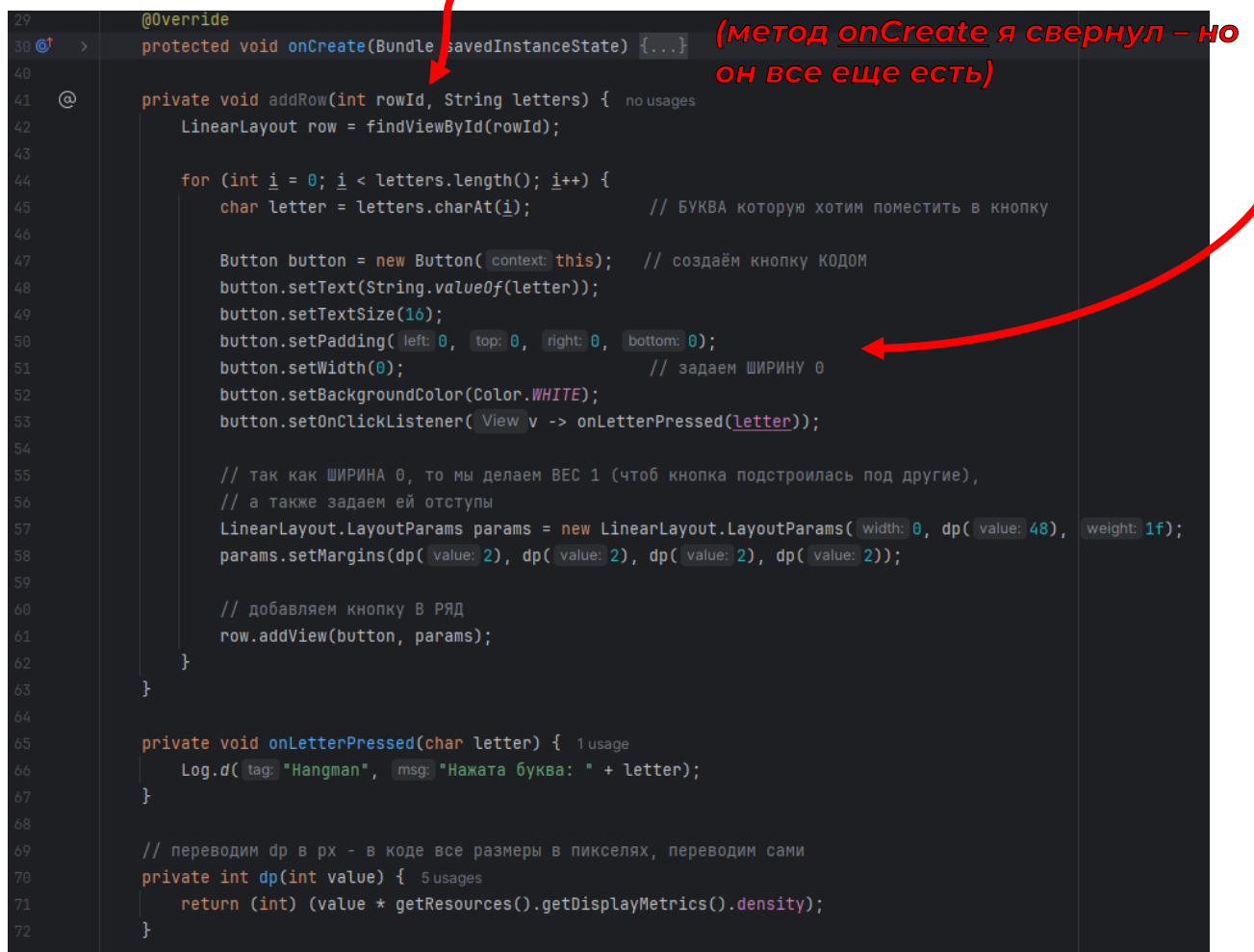
а зачем нам вообще **в коде** создавать кнопки, если мы можем **в XML** делать Buttons?

8 букв в ряду * 4 ряда = 32 кнопки. В XML это примерно **200 СТРОК ПОЧТИ ОДИНАКОВОГО ТЕКСТА**, в которых очень легко опечататься и сложно менять все разом.

А в **Java** – цикл выйдет примерно на **15 СТРОК** (разница очевидна же?), в котором опечататься негде: компилятор проверит всё сам. Если имеете дело с однотипными элементами – всегда цикл (обычно так, но не всегда).

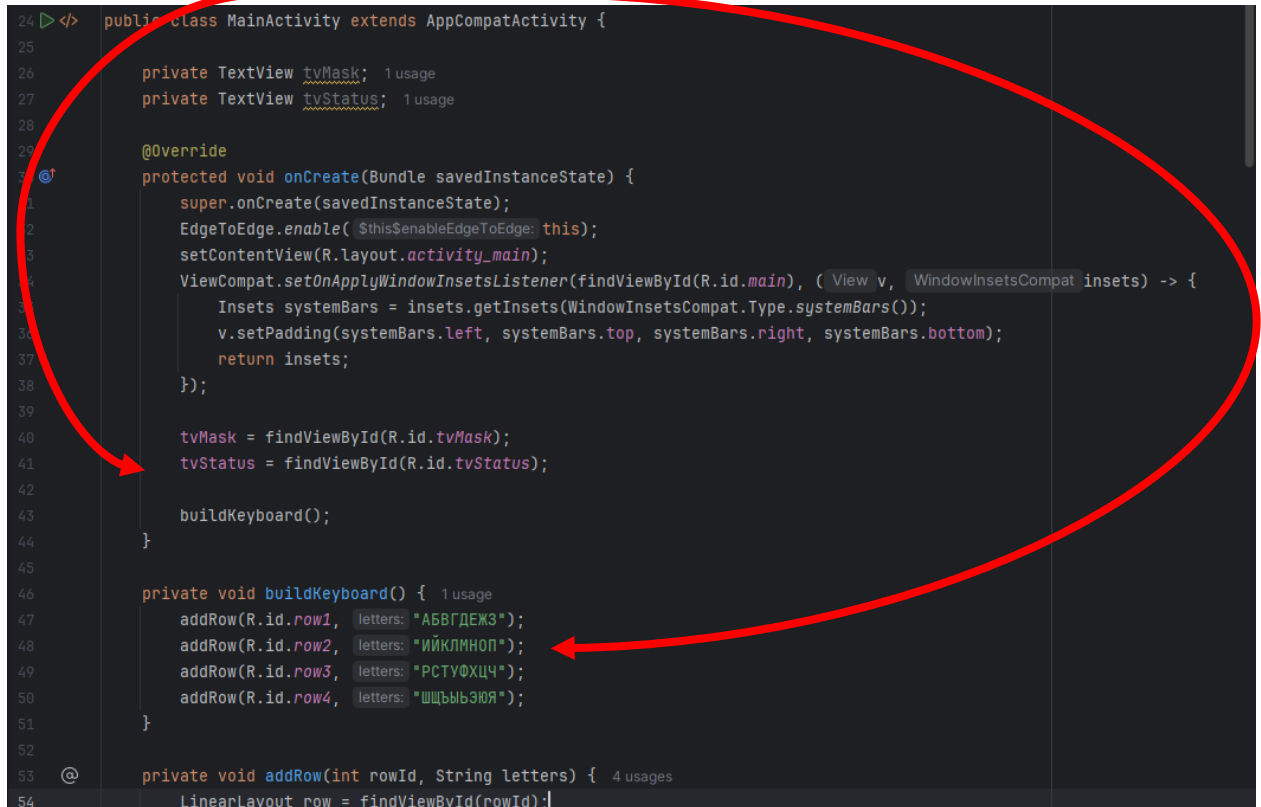
Алфавит разобьём на 4 строки по 8 букв. **Букву Ё не используем** – в играх и кроссвордах Е и Ё традиционно притесняют, получается ровно $32 = 4 \times 8$.

Откройте **MainActivity**, создадим **метод**, который будет создавать **ОДНУ кнопку** и добавлять ее **в РЯД**:



```
29 @Override
30 > protected void onCreate(Bundle savedInstanceState) {...} (метод onCreate я свернул – но он все еще есть)
31
32 private void addRow(int rowId, String letters) { no usages
33     LinearLayout row = findViewById(rowId);
34
35     for (int i = 0; i < letters.length(); i++) {
36         char letter = letters.charAt(i); // БУКВА которую хотим поместить в кнопку
37
38         Button button = new Button(context: this); // создаём кнопку КОДОМ
39         button.setText(String.valueOf(letter));
40         button.setTextSize(16);
41         button.setPadding(left: 0, top: 0, right: 0, bottom: 0);
42         button.setWidth(0); // задаем ШИРИНУ 0
43         button.setBackgroundColor(Color.WHITE);
44         button.setOnClickListener(View v -> onLetterPressed(letter));
45
46         // так как ШИРИНА 0, то мы делаем ВЕС 1 (чтоб кнопка подстроилась под другие),
47         // а также задаем ей отступы
48         LinearLayout.LayoutParams params = new LinearLayout.LayoutParams(width: 0, dp( value: 48), weight: 1f);
49         params.setMargins(dp( value: 2), dp( value: 2), dp( value: 2), dp( value: 2));
50
51         // добавляем кнопку в РЯД
52         row.addView(button, params);
53     }
54 }
55
56 private void onLetterPressed(char letter) { 1 usage
57     Log.d(tag: "Hangman", msg: "Нажата буква: " + letter);
58 }
59
60 // переводим dp в px - в коде все размеры в пикселях, переводим сами
61 private int dp(int value) { 5 usages
62     return (int) (value * getResources().getDisplayMetrics().density);
63 }
64 }
```

Допишем также **onCreate** и **метод** создания клавиатуры:



```
24 public class MainActivity extends AppCompatActivity {
25
26     private TextView tvMask; 1 usage
27     private TextView tvStatus; 1 usage
28
29     @Override
30     protected void onCreate(Bundle savedInstanceState) {
31         super.onCreate(savedInstanceState);
32         EdgeToEdge.enable(this);
33         setContentView(R.layout.activity_main);
34         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, WindowInsetsCompat insets) -> {
35             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
36             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
37             return insets;
38         });
39
40         tvMask = findViewById(R.id.tvMask);
41         tvStatus = findViewById(R.id.tvStatus);
42
43         buildKeyboard();
44     }
45
46     private void buildKeyboard() { 1 usage
47         addRow(R.id.row1, letters: "АБВГДЕЖЗ");
48         addRow(R.id.row2, letters: "ИЙКЛМНОП");
49         addRow(R.id.row3, letters: "РСТУФХЦЧ");
50         addRow(R.id.row4, letters: "ШЩЬЫЪЗЮЯ");
51     }
52
53     private void addRow(int rowId, String letters) { 4 usages
54         LinearLayout row = findViewById(rowId);
```

Разберём **новые приемы**:

1. **LinearLayout.LayoutParams(0, dp(48), 1f)** – это те же свойства, что мы писали в XML = **(layout_width, layout_height, layout_weight)**.
2. **setWidth(0)** – у стандартной кнопки есть **ширина по умолчанию** из темы (около 64dp чтобы текст помещался).
Представьте – восемь кнопок по 64dp = **512dp**, а экран примерно **360dp**: ряд не влезет, кнопки вылезут за край.
Потому мы задаём кнопкам ширину при помощи Weight, поэтому минимум отключаем.

Запустите. Потыкайте буквы и проверьте по **Logcat** (фильтр tag:Hangman), что каждая нажатая буква доходит до кода. Интерфейс готов.

Требования к итоговой версии

1. **Экран:** заголовок, виселица, маска слова (моноширинный шрифт), счётчик попыток, клавиатура 4×8.
2. Нажатие **переключает** этапы 0 -> 6
3. Клавиатура **создана циклом из кода**
4. Нажатия пишутся в **Logcat**
5. Строки – в **ресурсах**

Дополнительно (по желанию)

1. **Скруглённые клавиши.** В одной из лабораторных вы делали shape-ресурсы: создайте <shape> со скруглёнными углами и белой заливкой, назначьте кнопке `button.setBackgroundResource(R.drawable.key_bg);` вместо `setBackgroundColor`
2. **Глаза *_* на последнем этапе.** Если `stage >= 6`, дорисуйте на голове две пары коротких линий-крестиков